MISR UNIVERSITY
FOR SCIENCE & TECHNOLOGY
College of Information Technology

CS312 – MODERN DEVELOPMENT TOOLS | COURSE PROJECT

Masareefy

Expense Tracking System – Project Documentation

Student ID	Name
200061220	Mohamed Yousry
200052397	Malak Magdy
200051665	Malak Hany
200054010	Sama Ehab
200050898	Hla Azar
200052416	Jana Mohammed

Under the supervision of
Dr. Nehal Abdel Salam
Eng. Mariam Farid

Version 1.0 | October 2026

1. Introduction

1.1 The problem

Many people do not know where their money goes. Expenses are written on paper or in chat messages, there is no grouping, no spending limit and no clear picture of the month. **Masareefy** (Arabic for "my expenses") is a simple Windows program that solves this.

1.2 What Masareefy does

Masareefy lets the user record expenses, group them in colored categories, search and filter them, set a monthly budget for each category, and see reports and charts. If a few users run the program on the same network, a small server tells the others when someone adds an expense.

1.3 Scope

Included	Not included
Login and register, categories, expenses, search and filter, budgets, reports, pie and bar charts, CSV export, simple server and clients, error log	Web or mobile version, online hosting, bank connection, encrypted network traffic, multiple currencies

1.4 Technology

Part	What we use
Language	C# (.NET) with Windows Forms
Database	SQLite (one file, no server to install) using ADO.NET
Charts	GDI+ (System.Drawing) – we draw the charts ourselves
Background work	async/await for the CSV export
Network	TCP sockets (System.Net.Sockets) – one server, many clients
Code sharing	Git and GitHub

2. Requirements

2.1 Functional requirements

ID	Requirement
FR-01	A user can register and log in with a username and password.
FR-02	Passwords are saved as hashes, not as plain text.
FR-03	Add, edit, delete and list categories. Each category has a name, a color and a monthly budget.
FR-04	Add, edit, delete and list expenses (amount, category, date, description, notes).

ID	Requirement
FR-05	Search expenses by keyword.
FR-06	Filter expenses by date range and category, and show the number of rows and the total.
FR-07	Check all input before saving and show a clear message beside the wrong field.
FR-08	Show a warning when a category reaches 80% of its budget and an alert at 100%.
FR-09	Dashboard: total for this month, top category, budget status.
FR-10	Reports: monthly total, spending by category, budget versus spent.
FR-11	Pie chart (spending by category) and bar chart (total per month), drawn with GDI+.
FR-12	Export the expenses to a CSV file in the background with a progress bar, so the window does not freeze.
FR-13	A simple server accepts several clients. When one client adds an expense, the other clients get a message.
FR-14	Catch errors, write them to a log file and show a friendly message instead of crashing.

2.2 Non-functional requirements

Quality	Requirement
Easy to use	A new user can add the first expense in about two minutes without help.
Responsive	The window never freezes while exporting.
Reliable	No crash on wrong input or on a missing server; errors are logged.
Safe	Hashed passwords; SQL commands always use parameters; a user sees only their own data.
Clean code	Three layers (screens, logic, data) and clear names, so the team can work on the same code easily.
Runs anywhere	Windows 10 and 11, no database installation.

2.3 Rules

- Amount must be greater than zero.
- Category name is unique for each user.
- A category that has expenses cannot be deleted.
- Budget status: normal below 80%, warning from 80%, exceeded at 100%.

3. Course Technical Requirements

The course asks for a project that is more than a simple CRUD program. This table shows how Masareefy covers each required point.

#	Course requirement	How Masareefy covers it
1	GUI	Login, Dashboard, Expenses, Categories and Reports screens
2	Windows Forms	DataGridView, ComboBox, DateTimePicker, TextBox, ProgressBar, ErrorProvider, menu

#	Course requirement	How Masareefy covers it
3	Good C#	Classes, interfaces, three layers, async/await, LINQ, clear naming
4	CRUD	Categories and expenses
5	Database connectivity	SQLite with ADO.NET and parameterized commands
6	Reports / Statistics	Monthly total, by category, budget versus spent
7	Exception handling	try/catch, log file, friendly messages
8	Validation	Rules in Section 2.3 and checks on every form
9	Search / Filter	Keyword, date range and category
10	GDI+ graphics	Pie chart and bar chart drawn by hand
11	Multithreading / async	CSV export in the background (one function, as required)
12	Networking	Simple TCP server and clients that exchange messages

4. Design

4.1 How the program is built

The program has three layers. Screens only show data and take input. Services hold the rules. Repositories talk to the database. This keeps the code organized and easy to share.

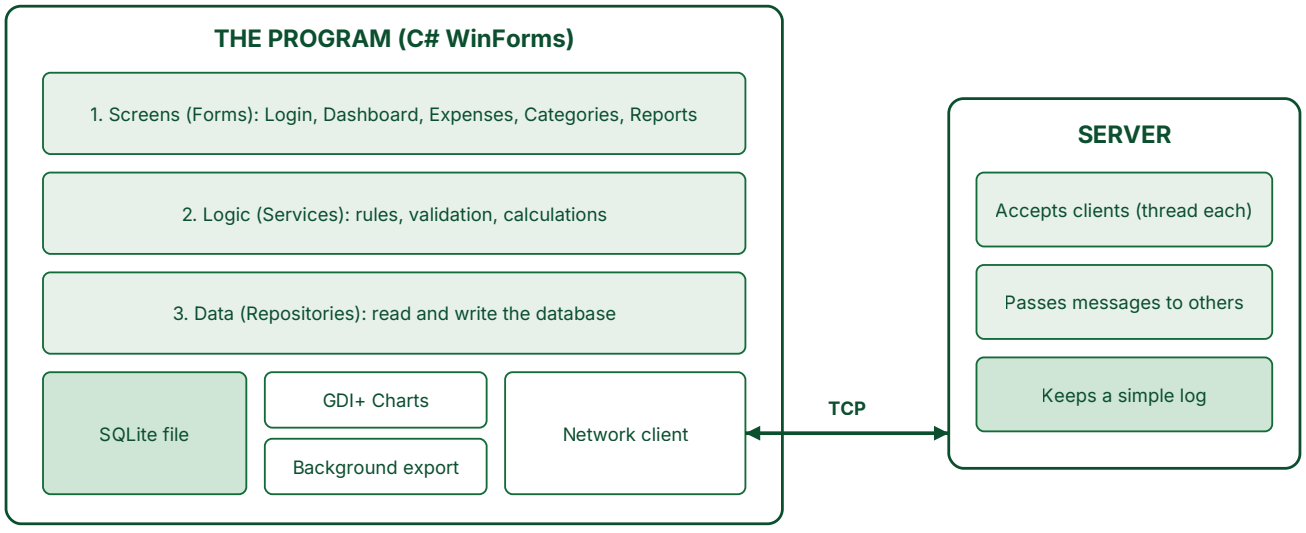


Figure 1 – Program layers and the server

4.2 Database

Three tables are enough for Masareefy.

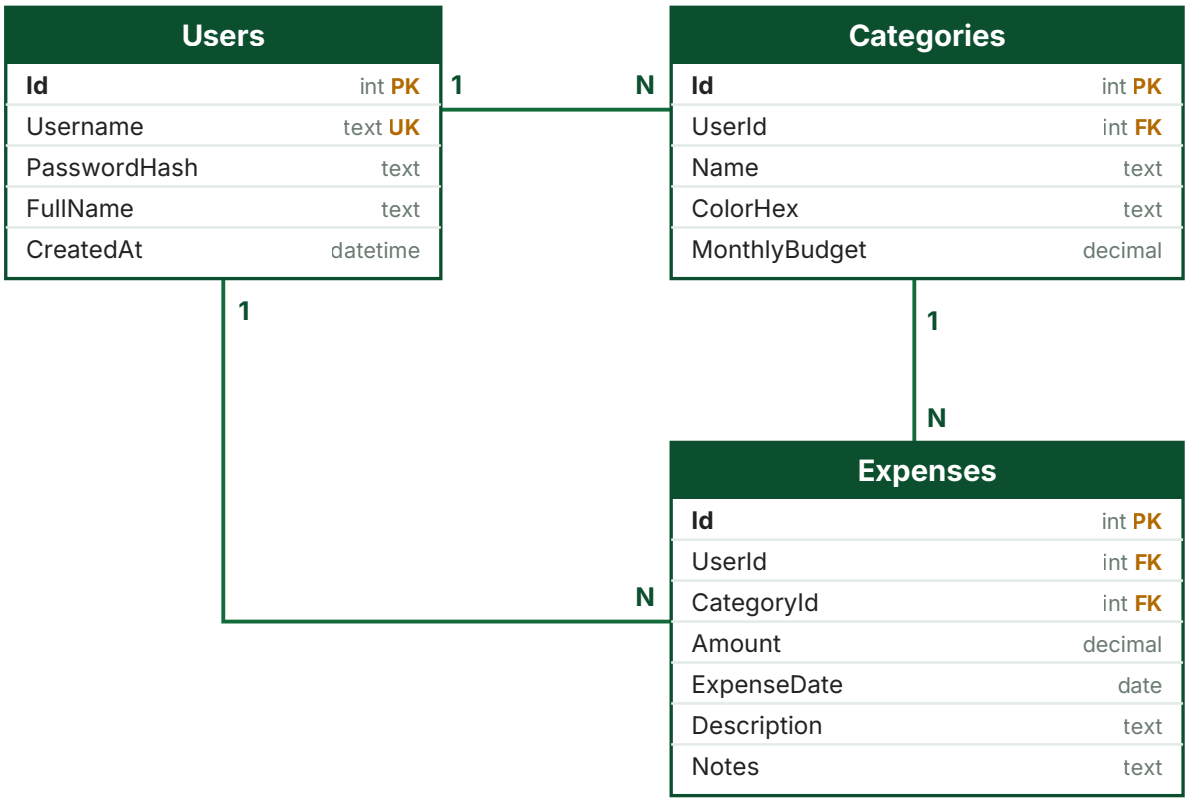
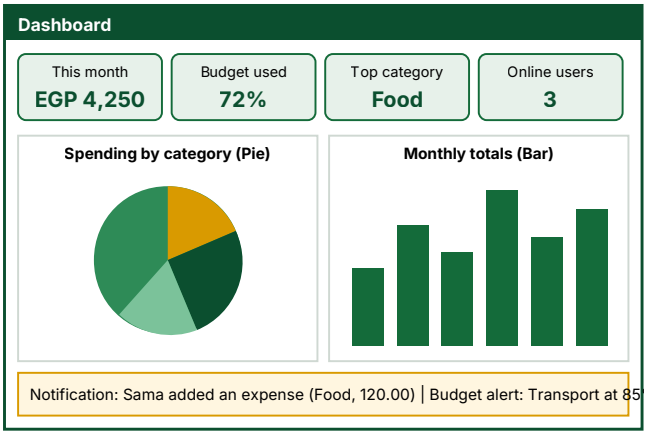


Figure 2 – Database tables (PK = primary key, FK = foreign key, UK = unique)

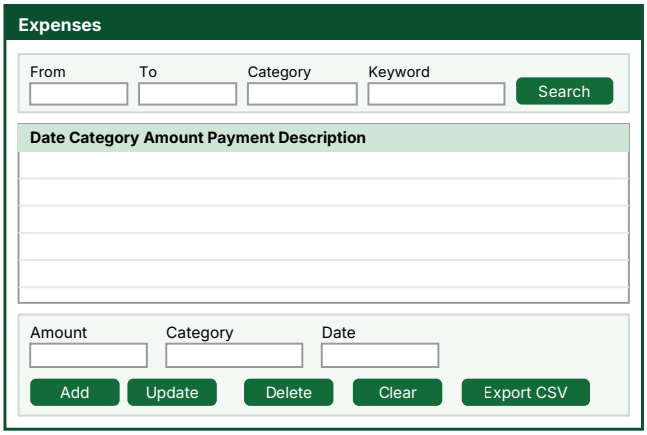
Table	Purpose
Users	Accounts: username, password hash, full name
Categories	Name, color for charts, monthly budget; belongs to a user
Expenses	Amount, date, description, notes; belongs to a user and a category

4.3 Screens

Screen	What it does
Login / Register	Create an account or sign in
Dashboard	This month’s total, top category, budget status, notifications from other users
Expenses	List, add, edit, delete, search and filter, export to CSV
Categories	Manage categories, colors and budgets
Reports	Choose a report and a month, see the table and the charts
Server window	Start and stop the server and see who is connected



Dashboard (sketch)



Expenses screen (sketch)

4.4 Charts, background export and network

Part	How it works
Charts (GDI+)	A custom control draws the pie chart with `FillPie` (each slice angle = value / total x 360, color from the category) and the bar chart with `FillRectangle` and `DrawString`.
Background export	The export button starts an `async` method that writes the CSV line by line and reports progress to a progress bar. The window stays usable.
Network	The server listens on one port and starts one thread for each client. Messages are single text lines, for example `EXPENSE_ADDED|sama|Food|120`. The server sends the line to all other clients, and each client shows it as a notification. If the server is not running, Masareefy still works normally.

4.5 Errors and validation

Field	Rule	Message
Username	Required, unique	Username already exists
Password	At least 6 characters	Password is too short
Amount	Number greater than 0	Amount must be greater than zero
Category	Must be chosen	Please choose a category
Description	Required	Description is required
Date range	From date not after to date	Start date must be before end date

Every button handler uses `try/catch`. Errors are written to a log file with the time and message, and the user sees a simple message. The program never closes because of an error.

5. Work Plan

5.1 How we work

> All six of us work together on every part of the project. Nobody owns a single screen or a single topic, so everybody learns everything and nobody is stuck waiting for someone else.

1. We meet (in person or online) and pick the next feature.
2. We agree together how it should look and work.
3. One person types while the others watch and help, then we swap.
4. We test the feature together.
5. We push it to GitHub.

5.2 Simple GitHub rules

- One repository. `main` is the final version, `develop` is where we collect our work.
- Always `git pull` before you start and `git push` when you finish.
- Tell the group which form you are editing, so two people never edit the same form at the same time.
- Commit often with a short message that says what you did.
- Use the Visual Studio `.gitignore` file so `bin` and `obj` are not uploaded.

5.3 Timeline (8 weeks)

Task	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8
Plan, database design, project set-up	■							
Login and Expenses (add, edit, delete, list)	■	■						
Categories, validation, error handling			■	■				
Search and filter			■	■				
Reports and charts (GDI+)					■	■		
CSV export in the background					■	■		
Simple server and clients					■	■	■	
Testing everything							■	■
Documentation and presentation							■	■
Delivery		**D1**		**D2**		**D3**		**D4**

D1 to D4 are the four deliveries (every two weeks).

Delivery	Week	What we show
D1	2	Program opens, login works, expenses can be added, edited, deleted and listed from the database
D2	4	Categories with budgets, validation, error handling, search and filter
D3	6	Reports, pie and bar charts, background CSV export, first version of server and client
D4	8	Network finished, everything tested, documentation and presentation ready

5.4 Risks

Risk	What we do
We are still learning C#	Build one small step at a time and run the program after every step; help each other
Two people change the same file	Tell the group which form you work on; pull before you start
Network takes longer than planned	Keep it simple (text messages); the program works without the server
Work left to the last week	Test after every feature and keep to the four deliveries
Lost code	Everything is on GitHub

6. Testing

We test together after every feature and again before every delivery, on two different computers. The last column is for writing Pass or Fail while testing.

ID	What we test	Expected result	Result
TC-01	Register a new user	Account is created	
TC-02	Log in with correct data	Dashboard opens	
TC-03	Log in with a wrong password	Error message, no login	
TC-04	Add an expense with valid data	It appears in the list and totals update	
TC-05	Add an expense with amount 0 or negative	Rejected with a message	
TC-06	Add an expense without a category	Rejected with a message	
TC-07	Edit an expense	Changes are saved	
TC-08	Delete an expense	Asks to confirm, then removes it	
TC-09	Add a category with color and budget	Saved and shown in the list	
TC-10	Delete a category that has expenses	Not allowed, message shown	
TC-11	Search by keyword	Only matching rows are shown	
TC-12	Filter by date range and category	Correct rows, count and total	
TC-13	Monthly report	Total equals the manual sum	
TC-14	Pie and bar charts	Numbers match the report	
TC-15	Export CSV	Window stays responsive; file opens in Excel	
TC-16	Missing database file	Friendly message, log written, no crash	
TC-17	Start the server and connect two clients	Both show connected	
TC-18	One client adds an expense	The other client shows a notification	
TC-19	Stop the server	Client shows disconnected; program still works	

7. Project Rubric Checklist

This table follows the grading rubric from the course and shows where each part is covered.

Rubric component	Marks	Where in Masareefy
Problem Definition & Requirements	5	Sections 1 and 2
GUI Design & Usability	10	Section 4.3: five main screens with clear layout and messages
Windows Forms Controls & Event Handling	10	DataGridView, ComboBox, DateTimePicker, ProgressBar, ErrorProvider, button events
OOP / Code Organization	10	Section 4.1: three layers, classes and interfaces
Database Design & Connectivity	15	Section 4.2: SQLite, three tables, repositories
CRUD + Search + Filtering + Validation	15	FR-03 to FR-07
Exception Handling & Reliability	5	Section 4.5: try/catch and log file
GDI+ / Graphics / Visualization	5	Pie and bar charts (FR-11)
Multithreading / Async Feature	5	Background CSV export (FR-12)
Networking / Client-Server Feature	5	Simple server and clients (FR-13)
Reports / Statistics / Charts	5	FR-09 to FR-11
Testing & Application Quality	3	Section 6
Documentation	2	This document
Presentation & Demo	5	Week 8: we rehearse together